

Ministerul Educației și Cercetării al Republicii Moldova

Universitatea Tehnică a Moldovei

Facultatea Calculatoare, Informatică și Microelectronică

Departamentul Ingineria Software și Automatică

Report on laboratory work no. 13

Course: Operating Systems

Topic: Semaphores in Your Own Kernel

Done by:
st. gr. FAF-243, Ilie Covali

Checked by:
Reițman Patricia

Chișinău – 2026

Operating Systems · FCIM / FAF · UTM · 2026-2027

Lab 13 · Semaphores in Your Own Kernel

Metadata & Environment

- **Course:** Operating Systems, FCIM / FAF, UTM, Academic Year 2026-2027
- **Lab Work:** Lab 13 (Track B: Semaphores in Your Own Kernel, xv6)
- **Author:** Ilie Covali (st. gr. FAF-243)
- **Verified by:** Reițman Patricia
- **Host OS:** Linux (CachyOS x86_64, Kernel 7.3.1-1-cachyos)
- **Target Architecture:** RISC-V 64-bit (`riscv64-elf-gcc` 14.2.0)
- **Emulator:** QEMU System Emulator (`qemu-system-riscv64` 10.1.0)
- **Source Tree:** `/home/ilie/xv6-riscv`
- **Workspace:** `/home/ilie/Documents/SO/os-lab13` (`~/os-lab13`)

Purpose of the Work

1. Extend the xv6 RISC-V operating system kernel with kernel-level counting semaphores via three new system calls: `sem_init` (24), `sem_down` (25), and `sem_up` (26).
2. Trace the hardware privilege transition and system call dispatch mechanism from user space assembly stubs (`user/usys.pl`) across the RISC-V `ecall` trap vector into kernel dispatch tables (`kernel/syscall.c`).
3. Construct a kernel-space synchronization primitive incorporating spinlock-protected integer state counters and kernel sleep/wakeup channel queues.
4. Analyze two-stage non-busy waiting mechanisms (`sleep_prepare` and `sleep(void)`) to eliminate lost-wakeup race conditions while ensuring that processes release spinlocks before sleeping.
5. Validate kernel semaphore correctness using multi-stage synchronization suites (`user/semtest.c`), verifying temporal ordering enforcement and strict alternating ping-pong handshakes across child and parent processes.
6. Evaluate mutual exclusion guarantees under true symmetric multiprocessing (SMP on 3 virtual CPU cores) using unbuffered character-by-character UART output streams (`user/mix.c`).
7. Investigate critical section atomicity, spurious wakeups, predicate re-evaluation loops (`while` vs. `if`), and catastrophic spinlock deadlock conditions when sleeping inside the kernel.

Part 1. The Numbers and the User Side

1. System Call Architecture in xv6

In RISC-V xv6, user-space programs execute in unprivileged User Mode (U-mode). Requesting kernel synchronization requires transitioning into privileged Supervisor Mode (S-mode) via an environment call (`ecall`). The kernel reads the system call identifier from register `a7` and input parameters from registers `a0`, `a1`, etc.

We assign three unique system call identifiers in `kernel/syscall.h`: - `SYS_sem_init` (24): Initializes semaphore $id \in [0, 7]$ to starting capacity $value \geq 0$. - `SYS_sem_down` (25): Decrements semaphore id; if the value is 0, suspends the calling process until awakened. - `SYS_sem_up` (26): Increments semaphore id and awakens waiting processes.

```
#define SYS_sync      22
#define SYS_trace     23
#define SYS_sem_init  24
#define SYS_sem_down  25
#define SYS_sem_up    26
```

Figure 1.1 – System call number definitions in kernel/syscall.h

2. User Space Interface and Assembly Stubs

User programs access these primitives through prototypes defined in `user/user.h`:

```
int sem_init(int, int);
int sem_down(int);
int sem_up(int);
```

Figure 1.2 – User-space function prototypes in user/user.h

The Perl script `user/usys.pl` automatically generates the RISC-V assembly stubs in `user/usys.S`:

```
entry("sem_init");
entry("sem_down");
entry("sem_up");
```

Figure 1.3 – Assembly stub definitions in user/usys.pl

Each stub loads the respective constant into `a7`, traps into the kernel with `ecall`, and executes `ret` upon return:

```
.global sem_down
sem_down:
 li a7, SYS_sem_down
 ecall
 ret
```

Figure 1.4 – Generated RISC-V machine stub for sem_down

3. User Test Suite: `semtest.c`

We develop `user/semtest.c` implementing two validation stages: - **Test 1 (Temporal Ordering):** Semaphore 0 is initialized to 0. A child process executes `sem_down(0)` and must block until the parent sleeps for 10 timer ticks (`pause(10)`) and calls `sem_up(0)`. - **Test 2 (Ping-Pong Turn-Taking):** Semaphore 1 (ping's turn) is initialized to 1; semaphore 2 (pong's turn) is initialized to 0. Two concurrent processes strictly alternate turns three times.

```
#include "kernel/types.h"
#include "user/user.h"

int main(void) {
  // Test 1: order. Semaphore 0 starts at 0, so the child must wait.
  sem_init(0, 0);
  if (fork() == 0) {
    sem_down(0);
    printf("child: I ran after the parent\n");
    exit(0);
  }
  pause(10);
  printf("parent: done, I let the child go\n");
  sem_up(0);
  wait(0);

  // Test 2: ping-pong. Semaphore 1 = ping's turn, semaphore 2 = pong's turn.
  sem_init(1, 1);
  sem_init(2, 0);
  if (fork() == 0) {
    for (int i = 1; i <= 3; i++) {
      sem_down(2);
      printf(" pong %d\n", i);
      sem_up(1);
    }
    exit(0);
  }
  for (int i = 1; i <= 3; i++) {
    sem_down(1);
    printf("ping %d\n", i);
    sem_up(2);
```

```
  }
  wait(0);
  printf("semtest: OK\n");
  exit(0);
}
```

Figure 1.5 – Implementation of verification suite in user/semtest.c

4. Check 1: Syscalls Reach Kernel Before Implementation

Compiling `semtest` into `fs.img` before adding the kernel handlers verifies that user-space traps reach the kernel dispatch table, triggering unknown system call errors:

```
$ semtest
3 semtest: unknown sys call 24
4 semtest: unknown sys call 25
child: I ran after the parent
parent: done, I let the child go
```

Figure 1.6 – Check 1 trace demonstrating unhandled syscalls and inverted execution order

Without functioning semaphores, `sem_down(0)` returns -1 immediately without sleeping. Because the parent calls `pause(10)`, the child finishes *before* the parent. This inverted order confirms that user traps reach the kernel and demonstrates the necessity of the kernel implementation.

Part 2. The Kernel Side

1. Semaphore State Machine and Synchronization Design

Semaphores are maintained in a global kernel array `sems[NSEM]` where $NSEM = 8$. Each semaphore consists of: 1. `struct spinlock lock`: A hardware spinlock providing atomic test-and-set mutual exclusion around the integer counter. 2. `int value`: The resource counter representing available permits.

In xv6, uninitialized global variables reside in the BSS section and are zeroed on boot. A zeroed spinlock (`lock.locked == 0`) represents a free, unlocked spinlock.

2. Implementation: `kernel/sem.c`

```
#include "types.h"
#include "riscv.h"
#include "defs.h"
#include "param.h"
#include "spinlock.h"
#include "proc.h"

#define NSEM 8

struct sem {
  struct spinlock lock; // a zeroed spinlock is a free spinlock
  int value;            // how many may still pass
};

static struct sem sems[NSEM];

uint64 sys_sem_init(void) {
  int id, value;
  argint(0, &id);
  argint(1, &value);
  if (id < 0 || id >= NSEM || value < 0)
    return -1;
  acquire(&sems[id].lock);
  sems[id].value = value;
  release(&sems[id].lock);
  return 0;
```

```
}

uint64 sys_sem_down(void) {
  int id;
  argint(0, &id);
  if (id < 0 || id >= NSEM)
    return -1;
  struct sem *s = &sems[id];
  acquire(&s->lock);
  while (s->value == 0) {
    sleep_prepare(s);       // Register wait channel address
    release(&s->lock);      // Release spinlock before sleeping
    sleep();                // Block process; wake on wakeup(s)
    acquire(&s->lock);
    if (killed(myproc())) { // Support process termination
      release(&s->lock);
      return -1;
    }
  }
  s->value--;
  release(&s->lock);
  return 0;
}

uint64 sys_sem_up(void) {
  int id;
  argint(0, &id);
  if (id < 0 || id >= NSEM)
    return -1;
  struct sem *s = &sems[id];
  acquire(&s->lock);
  s->value++;
  wakeup(s);              // Wake all processes sleeping on channel s
  release(&s->lock);
  return 0;
}
```

Figure 2.1 – Kernel semaphore implementation in kernel/sem.c

3. Avoiding the Lost-Wakeup Race

In a naive implementation, releasing `s->lock` before calling `sleep()` creates a critical race condition: if another CPU executes `sem_up` and calls `wakeup(s)` in the gap between `release` and `sleep`, the wakeup signal is lost, leaving the sleeping process frozen permanently.

Our xv6 kernel resolves this via two-phase sleep: 1. `sleep_prepare(s)`: Acquires the process's `p->lock` and sets `p->chan = s` while holding `s->lock`. 2. `release(&s->lock)`: Releases the semaphore spinlock. 3. `sleep()`: Atomically transitions the process state to `SLEEPING` and calls `sched()`. If `wakeup(s)` intervened, `p->chan` was already cleared to 0, causing `sleep()` to return immediately without suspending.

4. Integration into xv6 Build System

We register the handlers in `kernel/syscall.c`:

```
extern uint64 sys_sem_init(void);
extern uint64 sys_sem_down(void);
extern uint64 sys_sem_up(void);

static uint64 (*syscalls[])(void) = {
  ...
  [SYS_sem_init] = sys_sem_init,
  [SYS_sem_down] = sys_sem_down,
  [SYS_sem_up]   = sys_sem_up,
};
```

Figure 2.2 – Syscall dispatch table entries in kernel/syscall.c

We append `$K/sem.o` to `OBJS` and the test binaries to `UPROGS` in `Makefile`.

5. Check 2: Validation under QEMU

Booting xv6 and executing `semtest` verifies complete correctness:

```
$ semtest
parent: done, I let the child go
child: I ran after the parent
ping 1
 pong 1
ping 2
 pong 2
ping 3
 pong 3
semtest: OK
```

Figure 2.3 – Execution output of semtest demonstrating correct ordering and alternating ping-pong

The child blocks inside `sem_down(0)` without consuming CPU cycles until the parent finishes its 10-tick delay and calls `sem_up(0)`. In Test 2, ping and pong alternate deterministically across three rounds.

Part 3. A Semaphore as a Lock: `mix`

1. Multi-Core SMP Concurrency in xv6

The QEMU virtual machine runs xv6 configured with 3 virtual CPU cores (`-smp 3`). When two processes execute concurrently, each process runs simultaneously on a distinct CPU core.

The test program `user/mix.c` spawns two processes: - Parent: Outputs 5 lines of 30 `A` characters (`AAAAAAAAAAAAAAAAAAAAAAAAAAAAAA\n`). - Child: Outputs 5 lines of 30 `b` characters (`bbbbbbbbbbbbbbbbbbbbbbbbbbbbbb\n`). - Each line is emitted character-by-character using individual `write(1, p, 1)` system calls.

```c
#include "kernel/types.h"
#include "user/user.h"

void lines(char *s, int uselock) {
  for (int k = 0; k < 5; k++) {
    if (uselock)
      sem_down(3);
    for (char *p = s; *p; p++)
      write(1, p, 1);
    if (uselock)
      sem_up(3);
  }
}

int main(int argc, char *argv[]) {
  int uselock = (argc > 1);
  sem_init(3, 1);
  if (fork() == 0) {
    lines("bbbbbbbbbbbbbbbbbbbbbbbbbbbbbb\n", uselock);
    exit(0);
  }
  lines("AAAAAAAAAAAAAAAAAAAAAAAAAAAAAA\n", uselock);
  wait(0);
  exit(0);
}
```

Figure 3.1 – Concurrency stress test implementation in user/mix.c

2. Execution Traces

Running `mix` without synchronization permits both CPU cores to write characters concurrently to the UART console:

```
$ mix
AAAbAbAbbbAbbAbAbAbAbAbAbAbAbAbAbAbAbAbAbAbAbAbAbAAbAbAbAbAbAbAb
AbAbAbAbAbAbAAAbAbAbAbAbAbAbAbAbAbAbAbbAbAbAbAbAbAbAbAbAbAbAbAb
bA
AbAbAbAbAbAbAbAbAbAbAbAbAbAbAbAbAbAbbAbAbAbAbAbAbAbAbAbAbAbAbAb
AbAbAbAbAbAbAbAbAbAbAbAbAbAbAbAbAbAbAbAbAbAbAbbAAbAbAbAbAbAAAbb
```

Figure 3.2 – Output of mix without lock: character streams corrupted across SMP cores

Running `mix lock` activates semaphore 3 (initialized to 1), enforcing strict mutual exclusion around each 31-character line:

```
$ mix lock
AAAAAAAAAAAAAAAAAAAAAAAAAAAAAAA
bbbbbbbbbbbbbbbbbbbbbbbbbbbbbbb
bbbbbbbbbbbbbbbbbbbbbbbbbbbbbbb
bbbbbbbbbbbbbbbbbbbbbbbbbbbbbbb
bbbbbbbbbbbbbbbbbbbbbbbbbbbbbbb
bbbbbbbbbbbbbbbbbbbbbbbbbbbbbbb
AAAAAAAAAAAAAAAAAAAAAAAAAAAAAAA
AAAAAAAAAAAAAAAAAAAAAAAAAAAAAAA
AAAAAAAAAAAAAAAAAAAAAAAAAAAAAAA
AAAAAAAAAAAAAAAAAAAAAAAAAAAAAAA
```

Figure 3.3 – Output of mix lock: whole lines preserved without character interleaving

Observe (Part 3 Answers)

1. **Without the lock, why do the letters mix? (How many system calls does one line cost?)** One line consists of 30 alphanumeric characters followed by a newline byte (`\n`), costing exactly **31 independent `write(1, p, 1)` system calls**. Under multi-core SMP execution (3 CPUs in QEMU), process A and process B execute concurrently on independent CPU cores, and kernel timer interrupts preempt execution at arbitrary instruction boundaries. As a consequence, the 31 system calls of both processes interleave non-deterministically on the shared UART console hardware buffer.

2. **With the lock every line is whole. Which two lines of mix.c make one line a critical section?**

   - Line 174: `if (uselock) sem_down(3);` immediately prior to the per-character loop.
   - Line 178: `if (uselock) sem_up(3);` immediately following the newline write. Together, these lines ensure that the entire 31-byte output sequence executes under mutual exclusion.

3. **In sem_down the test is "while (s->value == 0)", not "if". Why can a process wake up and still find 0?** The kernel function `wakeup(s)` wakes up **all** processes sleeping on channel address `s`. If multiple processes are blocked waiting for semaphore `s` and another process invokes `sem_up` (incrementing `value` from $0 \rightarrow 1$), all sleeping processes awaken simultaneously and contend for `s->lock`. The first process to acquire `s->lock` decrements `value` back to 0. When subsequent awakened processes acquire `s->lock`, they find `s->value == 0`. Evaluating the predicate within a `while` loop forces them to detect that no permit is available and return to sleep.

4. **sem_down calls release(&s->lock) BEFORE sleep(). What would go wrong if it slept holding the spinlock?** Sleeping while holding a spinlock causes a fatal **system-wide kernel deadlock**. In xv6, acquiring a spinlock disables hardware interrupts on that CPU core. If a process enters sleep holding `s->lock`:

   - No other process on any CPU can acquire `s->lock` to invoke `sem_up(s)` and increment the semaphore counter.
   - Any other process or kernel thread on another CPU attempting to acquire `s->lock` enters an infinite busy-wait spin loop with interrupts disabled, permanently halting that CPU.
   - The sleeping process can never be awakened, causing an unrecoverable kernel hang.

Conclusions

In this laboratory work, we implemented kernel-level synchronization primitives within the xv6 RISC-V operating system: 1. **Full Syscall Lifecycle:** We traced the transition from user-space assembly stubs (`ecall`) into privileged Supervisor Mode dispatch tables (`kernel/syscall.c`), linking user prototypes to kernel implementations. 2. **Kernel Semaphore Construction:** We designed counting semaphores utilizing spinlocks for integer state protection and integrated xv6's two-stage sleep mechanism (`sleep_prepare` and `sleep`) to guarantee race-free process suspension. 3. **Non-Busy Waiting:** We confirmed that blocked processes release CPU execution resources completely, transitioning to `SLEEPING` state until explicitly signaled via `wakeup()`. 4. **Multi-Core Synchronization:** We demonstrated that on SMP hardware, fine-grained system call sequences require mutual exclusion locks to prevent interleaved I/O corruption, and verified that binary semaphores enforce atomic critical sections. 5. **Kernel Deadlock Prevention:** We analyzed spinlock constraints, proving why holding spinlocks across sleep transitions induces fatal deadlock in interrupt-disabled kernel contexts.